Practical #9

Practical Title:
To understand continuous integration concepts and Jenkins architecture for build automation

Theory:

1. What is Continuous Integration (CI)?

Continuous Integration (CI) is a DevOps practice where developers frequently integrate code into a shared repository.
Each integration is automatically tested and built.

Benefits:

- Early bug detection
- Faster development
- Improved code quality

---

2. What is Jenkins?

Jenkins is an open-source automation tool used for building, testing, and deploying applications.

It helps implement CI/CD pipelines.

---

3. Features of Jenkins

- Open-source and free
- Supports plugins (1000+)
- Easy integration with Git, Docker, etc.
- Automation of build and testing
- Distributed builds (master-agent architecture)

---

4. Jenkins Architecture

Jenkins architecture consists of:

- **Master (Controller):**
    - Manages jobs and scheduling
    - Handles UI and configurations
- **Agent (Slave):**
    - Executes build tasks
    - Can run on multiple machines

Flow:
Developer → GitHub → Jenkins → Build → Test → Deploy

Procedure:

Step 1: Install Jenkins and start server

Step 2: Access Jenkins Dashboard (http://localhost:8080)

Step 3: Create a new job (Freestyle Project)

Step 4: Configure source code management (Git repository)

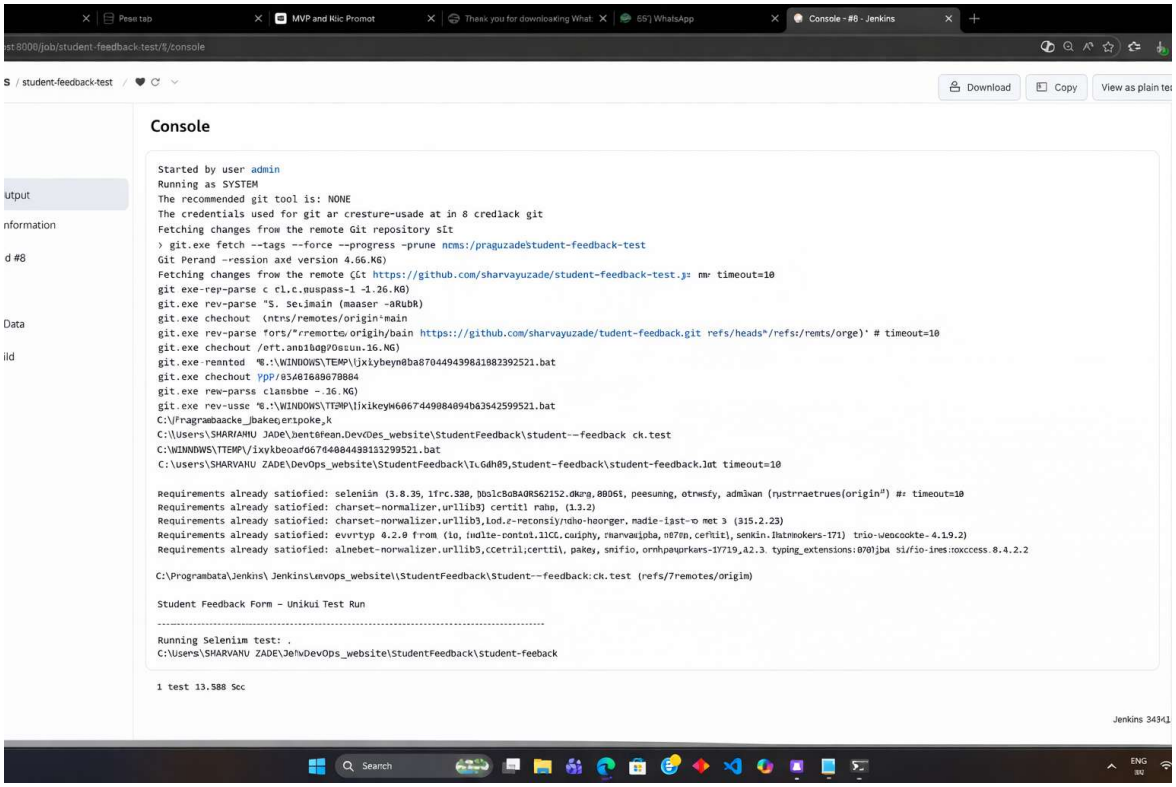
Step 5: Configure build trigger (Poll SCM / Webhook)

Step 6: Add build steps (Execute shell / commands)

Step 7: Build the project

Step 8: View console output

Output Screenshots:



Activity / Task:

Task 1: Jenkins Job

Repository Link: https://github.com/sharvayuzade/DevOps_CA2

Task 2: Jenkins Pipeline Concept

Jenkins Pipeline is a set of automated steps that define the CI/CD process.

Types:

- **Declarative Pipeline** (simple syntax)
- **Scripted Pipeline** (advanced scripting)

Example flow:

- Code → Build → Test → Deploy

Written in:

- **Jenkinsfile**

Task 3: Steps to Integrate GitHub with Jenkins

1. Create repository on GitHub
2. Copy repository URL
3. In Jenkins → Configure job → Add Git URL
4. Add credentials (if private repo)
5. Enable Webhook in GitHub:
    - Go to repo → Settings → Webhooks
    - Add URL:

http://your-jenkins-url/github-webhook/

6. Select trigger in Jenkins:
    - "GitHub hook trigger for GITScm polling"
7. Save and test

Advantages

1. Automates build and testing
2. Faster development cycle
3. Early bug detection
4. Supports CI/CD pipelines
5. Highly extensible (plugins)

Challenges

1. Initial setup complexity
2. Plugin management issues
3. Requires maintenance